ภาพรวมของบริการ (Service Overview)

IncidentPrioritization Service

1. Service Owner

นาย พชร พรพงศ์ 6609650509 ภาคพิเศษ

2. Service Purpose

IncidentPrioritization Service เป็นบริการที่รับผิดชอบการจำแนกประเภทภัยพิบัติและจัดกลุ่มระดับความเร่งด่วน (urgency level / priority group) ของเหตุการณ์ เพื่อสนับสนุนทีมกู้ภัยในการตัดสินใจเลือกเหตุการณ์ที่ควรเข้าช่วยเหลือก่อน โดยใช้ rule-based decision logic ที่ชัดเจนและโปร่งใส

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติ มักมีเหตุการณ์จำนวนมากถูกแจ้งเข้ามาพร้อมกัน แต่ไม่มีการจัดกลุ่มความเร่งด่วนที่ชัดเจน ทำให้หน่วยกู้ภัยไม่สามารถตัดสินใจได้อย่างรวดเร็วว่าเหตุการณ์ใดควรได้รับการช่วยเหลือก่อน บริการนี้จึงช่วยแปลงข้อมูลเหตุการณ์ให้เป็นระดับความเร่งด่วนที่เข้าใจง่าย เพื่อสนับสนุนการตัดสินใจภาคสนาม

3. Target Users

- Rescue Team (ทีมกู้ภัย)
- Dispatch Service (บริการที่ช่วยการมอบหมายหน่วยกู้ภัยหรือทรัพยากรไปยังเหตุการณ์ที่เกิดขึ้น)
- Command Center Dashboard (หน้าจอแสดงผลส่วนกลางสำหรับศูนย์ควบคุมเหตุการณ์)

บริการนี้ไม่ได้ออกแบบให้ประชาชนเรียกใช้โดยตรง แต่เป็น backend service สำหรับระบบตอบสนองภัยพิบัติ

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
    - จำแนกประเภทภัยพิบัติ (disaster_type)
    - ประเมินระดับความเร่งด่วน (urgency_level: critical/high/medium/low)
    - จัดกลุ่มลำดับความสำคัญ (priority_group: P1–P4)
    - จัดเก็บผลการ classify
    - อัปเดตระดับความเร่งด่วนเมื่อสถานการณ์เปลี่ยน

- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
    - การรับแจ้งเหตุและจัดเก็บ incident master data
    - การสั่งการหรือ dispatch หน่วยกู้ภัย
    - การจัดสรรทรัพยากรหรือยานพาหนะ
    - การจัดการศูนย์พักพิง

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การกำหนด disaster_type
- การกำหนด urgency_level
- การกำหนด priority_group (P1–P4)

การตัดสินใจอิงจาก:

- เนื้อหา description ของ incident
- การมีผู้บาดเจ็บหรือผู้ติดอยู่
- จำนวนผู้ได้รับผลกระทบ
- ประเภทสถานที่ (เช่น โรงพยาบาล โรงเรียน)
- ประเภทภัยพิบัติ

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด <u>โดยไม่ต้องรอ/ต้องรอ</u>การอนุมัติจากมนุษย์ในกรณีปกติ

6. Owned Data

- Classification Record
  ข้อมูลผลการจำแนก (disaster_type, urgency_level, priority_group) เป็นผลลัพธ์ของ decision logic ของบริการ จึงต้องอยู่ภายใต้บริการนี้
- Classification Status
  สถานะของการประเมิน (pending/classified/updated) เป็น operational state ที่เกี่ยวข้องโดยตรงกับ workflow ของบริการ
- Audit Fields
  created_at, updated_at, classified_by ใช้เพื่อการตรวจสอบย้อนหลัง (auditability) และความโปร่งใสในการตัดสินใจ

7. Linked Data (Reference Only)

ก่อนทำการ classify ระบบอาจอ้างอิงข้อมูลจาก Incident Service เช่น:

- incident_Id จาก Central Incident Schema
- description ของเหตุการณ์
- สถานะ incident (ACTIVE / RESOLVED)

บริการนี้ไม่เก็บสำเนาข้อมูล incident แบบถาวร แต่ใช้ incident_id เป็น reference เท่านั้น

8. Non-Functional Requirements

- ใช้ synchronous REST API (request–response)
- ต้องรองรับ concurrent request โดยไม่สร้างข้อมูลซ้ำ (incident_id ต้อง unique)
- ต้องมี idempotency เมื่อมีการเรียก classify ซ้ำ
- หาก Incident Service ไม่ตอบสนอง:

    - retry 1 ครั้ง
    - หากยังล้มเหลว ให้ตอบ error พร้อม reasonCode
- ระบบต้องไม่ crash เมื่อได้รับ request ที่ schema ไม่ถูกต้อง
- ต้องมี authentication สำหรับการเรียก API
  การ update พร้อมกันต้องป้องกัน race condition (เช่น ใช้ transaction หรือ optimistic locking)
- การตัดสินใจต้องโปร่งใสและสามารถอธิบาย rule ได้ (decision transparency)

Synchronous Function Contract

Base URL: http:IncidentPrioritizationService.com

API Contract #1: Classify Incident

ข้อมูลทั่วไป

- **Name:** Classify incident
- **Method:** POST
- **Path:** /v1/classifications
- **Type:** Synchronous

คำอธิบาย

- ใช้จำแนกประเภทภัยพิบัติและกำหนดระดับความเร่งด่วน (urgency level / priority group) เพื่อให้ Rescue Team สามารถตัดสินใจลำดับการช่วยเหลือได้

Request

Path/Query Params

- ไม่มี

Headers

- Content-Type: application/json
- Authorization: Bearer `<token>`

Body:

```
{
  "incident_Id": "uuid",
```

```
  "description": "string"
}
```

Response

Success: 200

```
{
  "incident_Id": "uuid",
  "disasterType": "fire",
  "urgencyLevel": "critical",
  "priorityGroup": "P1",
  "status": "classified"
}
```

Error: 400

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "incident_Id and description are required",
    "traceId": "uuid"
  }
}
```

Error: 409

```
{
  "error": {
    "code": "ALREADY_CLASSIFIED",
    "message": "incident already classified",
```

```
    "traceId": "uuid"
  }
}
```

Dependency / Reliability

- ตรวจสอบ incident_Id กับ Incident Service (optional validation)
- Idempotent: หากเรียกซ้ำด้วย incidentId เดิม จะไม่สร้าง record ใหม่
- Timeout: 10s
- Retry 1 ครั้ง หาก Incident Service ไม่ตอบสนอง

API Contract #2: Get Incidents from IncidentReporter Service

ข้อมูลทั่วไป

- **Name:** Get incidents from IncidentReporter
- **Method:** GET
- **Path:** /external/incidents
- **Type:** Synchronous

คำอธิบาย

- ใช้ดึงข้อมูลเหตุการณ์ (incident_Id และ description) จาก IncidentReporter Service เพื่อใช้เป็น input สำหรับการทำ classify ภายใน IncidentPrioritization Service

Request

Path/Query Params

- (optional) status (string) – ใช้ filter เฉพาะ incident ที่ active

Headers

- Content-Type: application/json
- Authorization: Bearer `<token>`

Body: ไม่มี

Response

Success: 200


```
{
 "items": [
    {
"incident_Id": "uuid",
"description": "fire at hospital"
    }
]
}
```


Error: 400


```
{
   "error": {
   "code": "INVALID_QUERY",
   "message": "invalid query parameter",
   "traceId": "uuid"
   }
}
```


Error: 503


```
{
 "error": {
"code": "DEPENDENCY_FAILED",
"message": "IncidentReporter Service unavailable",
"traceId": "uuid"
}
}
```

Dependency / Reliability

- เรียก IncidentReporter Service เพื่อดึงข้อมูล incident
- Timeout: 5–10s
- Retry 1 ครั้ง หาก service ไม่ตอบสนอง
- หากล้มเหลว: fallback ⟶ ใช้ข้อมูลจาก manual input หรือ cache แทน
- Read-only
- Idempotent

API Contract #3: Get Classified Incidents

ข้อมูลทั่วไป

- **Name:** List classified incidents by priority
- **Method:** GET
- **Path:** /v1/classifications
- **Type:** Synchronous

คำอธิบาย

- ใช้ดึงรายการเหตุการณ์ที่ถูก classify แล้ว เพื่อแสดงบน Dashboard หรือให้ Dispatch Service ใช้ตัดสินใจ

Request

Path/Query Param

- priorityGroup (string, optional)
- urgencyLevel (string, optional)

Headers

- Accept: application/json

Body: ไม่มี

Validation

- urgencyLevel ∈ {critical, high, medium, low}
- priorityGroup ∈ {P1, P2, P3, P4}

Response

Success: 200


```
{
  "items": [
    {
      "incident_Id": "uuid",
      "disasterType": "fire",
      "urgencyLevel": "critical",
      "priorityGroup": "P1"
    }
  ]
}
```


Error: 400


```
{
  "error": {
    "code": "INVALID_QUERY",
    "message": "invalid priorityGroup value",
    "traceId": "uuid"
  }
}
```


Dependency / Reliability

- ไม่เรียก service อื่น

- Read-only
- Idempotent
- Timeout: 10s
- ต้องรองรับ concurrent read

API Contract #4: Re-classify / Update Priority

ข้อมูลทั่วไป

- **Name:** Re-classify incident
- **Method:** PATCH
- **Path:** /v1/classifications/{incidentId}
- **Type:** Synchronous

คำอธิบาย

- ใช้ปรับปรุงระดับความเร่งด่วน (urgency level) และลำดับความสำคัญ (priority group) เมื่อมีข้อมูลใหม่ เช่น มีผู้บาดเจ็บเพิ่ม หรือมีข้อมูลจาก Alert Response Service เพื่อให้การตัดสินใจของ Rescue Team เป็นปัจจุบันมากขึ้น

Request

Path/Query Params

- incident_Id (string, required)

Headers

- Content-Type: application/json
- Authorization: Bearer `<token>`

Body:

```
{
 "description": "string (optional)",
 "severity_level": "critical",
 "note": "update due to new information"
}
```

Validation

- incident_Id ต้องไม่ว่าง
- incident_Id ต้องมีอยู่ในระบบ (ต้องเคยถูก classify มาก่อน)
- severity_level ∈ {low, medium, high, critical}
- ถ้ามี description ต้องไม่ว่าง
- ต้องมีอย่างน้อย 1 field สำหรับ update
- ไม่อนุญาตให้สร้าง record ใหม่ผ่าน API นี้

Response

Success: 200

```
{
  "incident_Id": "INC-2026-001",
  "urgencyLevel": "critical",
  "priorityGroup": "P1",
  "status": "updated"
}
```

Error: 400

```
{
  "error": {
  "code": "VALIDATION_ERROR",
  "message": "invalid input data",
  "traceId": "uuid"
  }
}
```

Error: 404

```
{
```

```
  "error": {
  "code": "NOT_FOUND",
  "message": "incident not found",
  "traceId": "uuid"
  }
}
```

Dependency / Reliability

- อาจเรียก Alert Response Service เพื่อดึงข้อมูลเพิ่มเติม (optional)
- Idempotent: การ update ด้วยข้อมูลเดิมจะไม่เปลี่ยนผลลัพธ์
- Timeout: 10s
- Retry: 1 ครั้ง หาก dependency ไม่ตอบสนอง
- ต้องรองรับ concurrent update (ใช้ optimistic locking หรือ transaction)

API Contract #5: Get Response Status (จาก AlertResponse Service)

ข้อมูลทั่วไป

- **Name:** Get alert response summary by incident
- **Method:** GET
- **Path:** /external/alerts
- **Type:** Synchronous

คำอธิบาย

- ใช้ดึงข้อมูลสรุปการตอบกลับของประชาชนจาก AlertResponse Service โดยอ้างอิงจาก incident_id เพื่อใช้เป็น input ในการปรับระดับความเร่งด่วนของเหตุการณ์

Request

Path/Query Param

- incident_id (string, required)

Headers

- Accept: application/json

Body: ไม่มี

Validation

- incident_id ต้องไม่ว่าง
- incident_id ต้องเป็น string ที่ถูกต้อง
- ต้องมี query parameter อย่างน้อย 1 ตัว (incident_id)
- response ต้องมีค่า total_need_help และ total_acknowledged

Response

Success: 200 (จาก Alert Service)


```
{
 "alert_id": "ALT-001",
"message_text": "Flood warning",
"total_sent": 100,
"total_acknowledged": 60,
"total_need_help": 10,
"status": "active"
}
```


Error: 400


```
{
```

```
  "error": {
    "code": "INVALID_QUERY",
    "message": "incident_id is required",
    "traceId": "uuid"
  }
}
```

Error: 503

```
{
  "error": {
    "code": "DEPENDENCY_FAILED",
    "message": "Alert Response Service unavailable",
    "traceId": "uuid"
  }
}
```

Dependency / Reliability

- เรียก AlertResponse Service
- Timeout: 5–10s
- Retry: 1 ครั้ง
- หากล้มเหลว: fallback ⟶ ใช้ rule-based อย่างเดียว
- Read-only
- Idempotent

ASynchronous Function Contract

Message contract #1: Publish Classified Incident

ข้อมูลทั่วไป

- **Message Name:** IncidentClassifiedEvent
- **Interaction Style:** Event Notification (fire-and-forget via Message Broker)
- **Producer:** IncidentPrioritization Service
- **Consumer:** Any subscribed services (e.g., Dashboard, Alert Service)
- **Channel/Queue:** incident.classified.events.v1
- **Version:** v1

คำอธิบาย

เมื่อ IncidentPrioritization Service ทำการ classify เหตุการณ์เสร็จแล้ว

จะทำการ publish event ไปยัง Amazon SNS

เพื่อให้ service อื่นสามารถ subscribe และนำข้อมูลไปใช้ต่อได้แบบ asynchronous

โดยผู้ส่งไม่ต้องรอผลลัพธ์จากผู้รับ

Request

Message Headers (required unless stated)

- messageType: IncidentClassifiedEvent
- messageId: UUID (ใช้เป็น idempotency)
- sentAt: ISO-8601 datetime
- traceId: string/uuid (optional)

Message Body

```
{
  "incident_Id": "INC0001",
  "disasterType": "fire",
  "urgencyLevel": "high",
  "priorityGroup": "P2",
  "status": "classified",
  "classifiedBy": "SYSTEM"
```

}

Field Definition:

Field	Type	Required	Description
incident_Id	string	Y	รหัสเหตุการณ์
disaster_type	string	Y	ประเภทภัยพิบัติ
urgencyLevel	enum	Y	CRITICAL / HIGH / MEDIUM / LOW
priorityGroup	enum	Y	P1 / P2 / P3 / P4
status	string	Y	สถานะของการ classify
classifiedBy	string	Y	SYSTEM หรือ OPERATOR

Validation Rules

1. incident_Id ต้องไม่ว่าง
2. urgencyLevel ∈ {CRITICAL / HIGH / MEDIUM / LOW}
3. priorityGroup ∈ {P1, P2, P3, P4}
4. messageId ต้องไม่ซ้ำ (idempotency check)
5. disaster_type ต้องเป็นค่าที่ระบบรองรับ

Response

ไม่มี response (fire-and-forget) - เนื่องจากเป็น Event-based communication

Producer จะไม่รอผลลัพธ์จาก consumer

Dependency / Reliability

- ใช้ SNS Topic เป็น message broker

- รองรับ multiple subscribers
- delivery แบบ at-least-once
- consumer ต้อง handle duplicate message เอง
- system ต้องไม่ crash หากไม่มี subscriber
- retry อัตโนมัติจาก SNS

Service Data

1) Incident Case Data (Owned by this service)

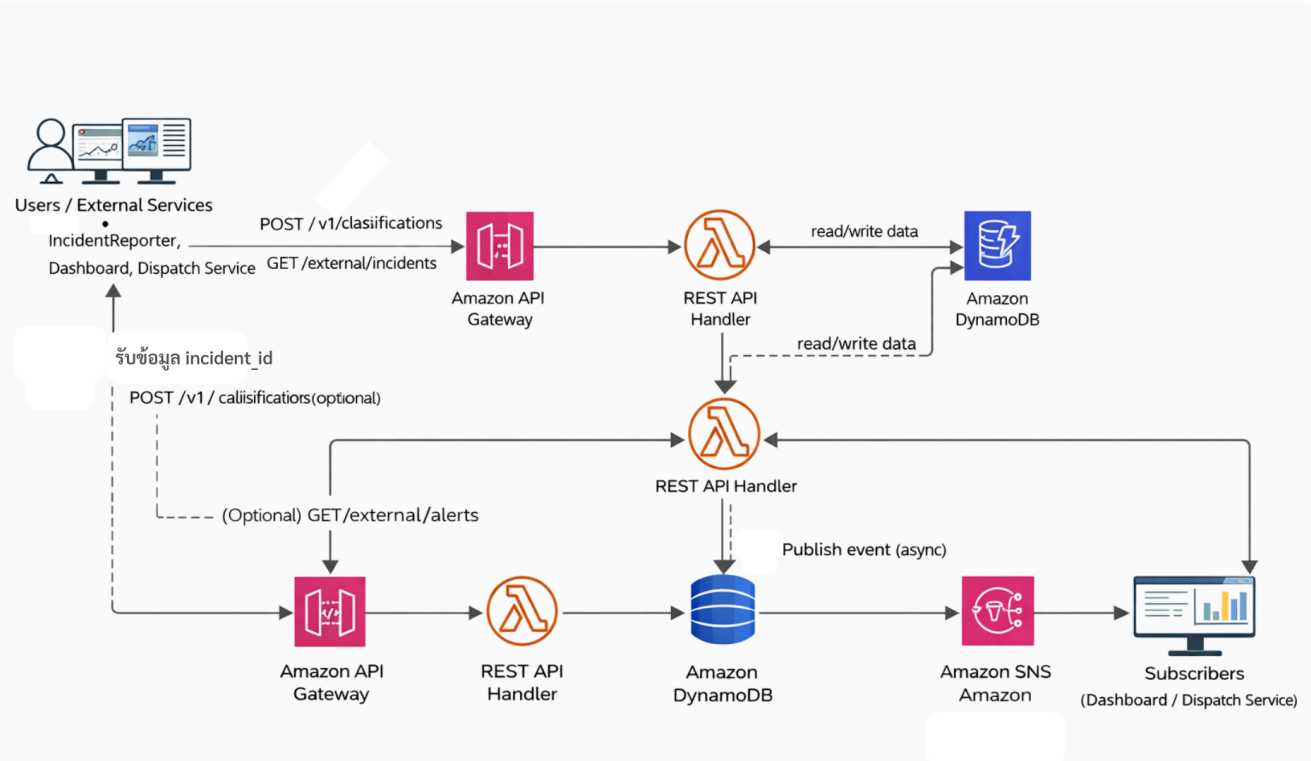
Field Name	Type	Required	Description	Example
caseId	string	Y (PK)	รหัสเคสเหตุการณ์	CASE001
incident_Id	uuid	Y (FK)	อ้างอิงเหตุการณ์หลัก	INC0001
disaster_type	enum	Y	ประเภทของภัยพิบัติ(FIRE / FLOOD /EARTHQUAKE / ACCIDENT / GENERIC)	Fire
description	string	N	รายละเอียดเหตุการณ์	Water level rising rapidly
severity_level	enum	Y	ระดับความรุนแรง (LOW / MEDIUM / HIGH / CRITICAL)	HIGH
impactScope	enum	Y	ขอบเขตผลกระทบ (INDIVIDUAL / COMMUNITY / CITY)	COMMUNITY
Vulnerable GroupFlag	boolean	Y	มีผู้เปราะบางที่เกี่ยวข้องหรือไม่	True
created_at	datetime	Y	วันที่สร้างข้อมูล	2026-03-01T10:05:00Z
updated_at	datetime	Y	วันที่แก้ไขล่าสุด	2026-03-01T10:10:00Z

2) Prioritization Result Data (Owned by this service)

Field Name	Type	Required	Description	Example
caseId	string	Y (FK)	อ้างอิง Incident Case	CASE001

urgencyLevel	enum	Y	ระดับความเร่งด่วน (LOW / MEDIUM / HIGH / CRITICAL)	HIGH
priorityGroup	enum	Y	กลุ่มลำดับความสำคัญ (P4 / P3 / P2 / P1)	P1
decisionReason	string	Y	คำอธิบายเหตุผลตาม rule base	High severity + vulnerable group
status	enum	Y	สถานะของการประเมิน (classified / updated)	สถานะของการประเมิน (classified / updated)
classifiedAt	datetime	Y	เวลาที่ระบบทำการจัดลำดับ	2026-03-01T10:11:00Z
lastUpdatedAt	datetime	Y	เวลาที่มีการอัปเดตล่าสุด	2026-03-01T10:15:00Z
classified_by	string	Y	SYSTEM หรือ OPERATOR	SYSTEM
reviewStatus	enum	Y	AUTO_APPROVED / PENDING_REVIEW	AUTO_APPROVED
version	integer	Y	ใช้สำหรับควบคุม concurrency (optimistic locking)	1

Service Architecture



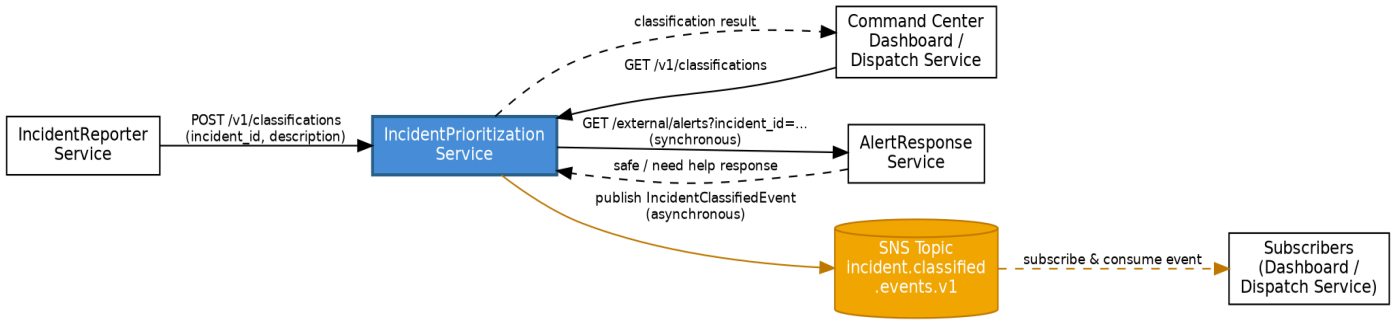
Components

- **User / External Services** : เป็นผู้เรียกใช้งาน API ของระบบเพื่อส่งหรือดึงข้อมูลเหตุการณ์
- **Amazon API Gateway**: รับ request แบบ REST (POST / GET)
- **Lambda REST API Handler:** ประมวลผล classify และบันทึกลง database และตอบกลับทันที
- **Amazon DynamoDB :** เก็บข้อมูล incident case data และ Prioritization Result Data
- **Amazon SNS (incident.classified.events.v1)** ใช้ publish event หลัง classify เพื่อให้ service อื่น subscribe ไปใช้งานต่อ
- **Subscribers (Dashboard / Dispatch Service)** รับ event ไปใช้ตัดสินใจส่งทีมช่วยเหลือหรือแสดงผล

Explanation

สถาปัตยกรรมของระบบนี้ประกอบด้วยสองรูปแบบการทำงานหลัก ได้แก่ การสื่อสารแบบ synchronous และ asynchronous โดยเริ่มจากรับข้อมูล incident (จาก API หรือดึงจาก IncidentReporter) เพื่อนำข้อมูลมาส่งต่อให้ Lambda เพื่อทำการ classify (disaster + priority) และดึงข้อมูลจาก Alert Response และ classify incident ใหม่ ข้อมูลตรงนี้ที่ได้มานำมาเพื่อช่วยเพิ่มการตัดสินใจในการเข้าช่วยเหลือ และต่อมาบันทึกข้อมูลลง DynamoDB ส่ง response กลับทันที (synchronous) แล้ว publish event ไปที่ SNS (asynchronous) และ service อื่นหรือหน่วยเข้าช่วยเหลือนำข้อมูลไปใช้ต่อได้

Service Interaction



Upstream Services (บริการต้นทางที่เรียกใช้งาน IncidentPrioritization Service)

- **IncidentReporter Service**

  ส่งข้อมูลเหตุการณ์ (incident_Id, description) ผ่าน API

  POST /v1/classifications แบบ synchronous

  เพื่อให้ IncidentPrioritization Service ทำการจำแนกประเภทภัยพิบัติและกำหนดระดับความเร่งด่วน

- **Command Center Dashboard / Dispatch Service**

  เรียกใช้งาน

  GET /v1/classifications แบบ synchronous

  เพื่อดึงข้อมูลเหตุการณ์ที่ถูกจัดลำดับแล้วไปใช้แสดงผลและช่วยตัดสินใจ

Downstream Services (บริการปลายทางที่ IncidentPrioritization Service เรียกหรือส่งข้อมูลไป)

- **AlertResponse Service**

  ถูกเรียกใช้งานแบบ synchronous ผ่าน

  GET /external/alerts?incident_id=...

  เพื่อดึงข้อมูลการตอบกลับของประชาชน (safe / need help)

  และนำมาปรับปรุงระดับความเร่งด่วนของเหตุการณ์ให้แม่นยำมากขึ้น

- **Amazon SNS (incident.classified.events.v1)**

  ใช้ publish event แบบ asynchronous

  เมื่อมีการ classify เหตุการณ์สำเร็จ (IncidentClassifiedEvent)

  เพื่อให้ service อื่นสามารถ subscribe และนำข้อมูลไปใช้งานต่อได้โดยไม่ต้องรอผลลัพธ์ทันที

- **Subscribers (Dashboard / Dispatch Service)**

  รับ event จาก SNS เพื่อนำข้อมูลไปใช้ เช่น: แสดงผลบน dashboard , สนับสนุนการตัดสินใจส่งทีมช่วยเหลือ

Link github : https://github.com/Phachara6609650509/CS366-Microservices-Disaster-Response---6609650509.git